# Implementation Plan - Phase 2: Computer Vision Feature Extraction & PyTorch Multi-Task Model Training

We are implementing **Phase 2** of the **AI-Based Precision Fertigation System**: Building the complete **OpenCV 24-Feature Extractor**, training a **PyTorch Multi-Task Learning (MTL) Neural Network**, and generating **SHAP Explainability (XAI)** weights for model predictions.

Project Location: `C:\Users\Varshani\.gemini\antigravity-ide\scratch\precision-fertigation-app`

---

## Technical Overview & Architecture

```
 [ RAW PLANT IMAGE ] ──► [ OPENCV 24-FEATURE EXTRACTOR ] ──► [ 24-FEATURE VECTOR ]
                                                                      │
 ┌────────────────────────────────────────────────────────────────────┘
 ▼
 [ PYTORCH MULTI-TASK NEURAL NETWORK ]
 ├── Shared Feature Representation Encoder (Dense + BatchNorm + ReLU)
 ├── Head 1: Growth Rate Prediction (Regression)
 ├── Head 2: Water Requirement Estimation (Regression - ml/day)
 ├── Head 3: Fertilizer Dosage Estimation (Regression - grams NPK)
 ├── Head 4: Disease Diagnosis (Classification - Healthy / Blight / Chlorosis)
 └── Head 5: Health Score Calculation (Regression - 0 to 100)
       │
       ▼
 [ SHAP EXPLAINER ENGINE ] ──► Computes feature importance weights for every prediction
```

---

## Proposed Changes

### 1. Computer Vision Feature Extractor (`backend/cv_engine/`)

#### [NEW] [feature_extractor.py](file:///C:/Users/Varshani/.gemini/antigravity-ide/scratch/precision-fertigation-app/backend/cv_engine/feature_extractor.py)
Build Python module using OpenCV & NumPy to process input plant photos and compute all **24 parameters**:
- **Morphological (1-10)**: Canopy area ($cm^2$), leaf area, leaf count, bounding height/width, projected area, node count, stem thickness estimation.
- **Color & Health (11-18)**: Excess Green Index ($ExG = 2G - R - B$), HSV Hue/Saturation means, RGB histograms, leaf color index, visible yellowing %, brown spot region %.
- **Texture & Disease (19-24)**: Gray-Level Co-occurrence Matrix (GLCM) contrast/energy, shape circularity, health indicator score, disease spot blob counting.

---

### 2. Multi-Task Learning & XAI Engine (`backend/ml_engine/`)

#### [NEW] [mtl_model.py](file:///C:/Users/Varshani/.gemini/antigravity-ide/scratch/precision-fertigation-app/backend/ml_engine/mtl_model.py)
PyTorch Neural Network architecture (`MultiTaskFertigationNN`) featuring:
- Shared Encoder: `Linear(24, 64) -> BatchNorm -> ReLU -> Dropout -> Linear(64, 32)`.
- 5 Specialized Heads for joint prediction of Growth, Water Need, Fertilizer Dose, Disease Diagnosis, and Health Score.

#### [NEW] [train.py](file:///C:/Users/Varshani/.gemini/antigravity-ide/scratch/precision-fertigation-app/backend/ml_engine/train.py)
Training script that:
- Generates/loads synthetic plant dataset feature matrices with target labels.
- Trains `MultiTaskFertigationNN` over 50 epochs using a multi-loss function ($L = \sum_{i=1}^5 w_i L_i$).
- Saves trained model weights to `backend/ml_engine/trained_models/mtl_fertigation_model.pth`.

#### [NEW] [xai_explainer.py](file:///C:/Users/Varshani/.gemini/antigravity-ide/scratch/precision-fertigation-app/backend/ml_engine/xai_explainer.py)
Module using `shap` or PyTorch gradient feature attributions to compute top positive and negative feature weights (e.g. *"ExG Index drove +40% of fertilizer recommendation"*).

---

### 3. FastAPI Ingestion & Inference Endpoint (`backend/routers/`)

#### [NEW] [inference.py](file:///C:/Users/Varshani/.gemini/antigravity-ide/scratch/precision-fertigation-app/backend/routers/inference.py)
New API route:
- `POST /api/telemetry/process-image`: Accepts plant image uploads, extracts 24 features via OpenCV, runs PyTorch MTL model inference, computes SHAP weights, saves records to SQLite database, and returns predictions to the frontend!

---

## Verification Plan

### Automated Tests
1. Execute `python backend/cv_engine/feature_extractor.py` on a sample plant image to verify extraction of all 24 parameters.
2. Execute `python backend/ml_engine/train.py` to train and save PyTorch model weights (`mtl_fertigation_model.pth`).
3. Run test script to verify `xai_explainer.py` generates valid SHAP weights.
4. Test image upload endpoint `POST /api/telemetry/process-image` via Python `requests`.

### Manual Verification
- Verify that live predictions and SHAP bars update on the **[http://localhost:3000](http://localhost:3000)** dashboard when an image is processed!